ENIGMA++

Luca Stefan Nicolae - S6816509
Artyom Kulimov - S6569862
l.nicolae@student.rug.nl & a.kulimov@student.rug.nl

October 28, 2026

1 Problem description

In this project, we will explore the implementation of three different pseudo-random number generators (random, LCG and subtractive) in the encryption and decryption process of an input text. Moreover, we will analyze the robustness of each in terms of how easily they can be brute forced in an attack.

2 Problem analysis

The three different ciphers work in the same way: a pseudo-random generator produces a sequence of bytes, and every character of the message is combined with one byte using this XOR operation:

```
encMessage[i] = message[i] ^ x;
```

Applying the XOR operation twice with the same byte gives the original value back; therefore, decryption consists of the same steps as encryption, the decoder generates the same bytes and combines them with the encoded message again. So, for example, with seed 42, the random generator produces byte 102 first, so, in this example, the letter H (72) is encoded as $72 \oplus 102 = 46$, and decoding it gives $46 \oplus 102 = 72$.

The receiver only needs to start the generator with the same key to reproduce the same sequence of bytes, which means that the security of the cipher depends solely on how difficult it is to guess the key. This is where the three generators differ:

- The random generator: The bytes are produced by `rand() % 256` after calling `srand(seed)`, and because the seed is a single byte, there are 2^8 possible keys, which means that it can be guessed quite fast.
- LCG: Every new byte depends on the previous one by multiplying it by a, adding c, and keeping only the last byte of the result ($x_{n+1} = (a \cdot x_n + c) \bmod 256$). This means that the key gives 2^{24} possible keys, which is more than the random generator, but is still possible for a computer to try all of them in a reasonable amount of time.
- Subtractive Generator: Instead of depending only on the previous byte, every next byte is calculated by subtracting the two earlier bytes from the sequence, the $(n-24)$th and $(n-55)$th byte. Because of this, the generator starts with 55 bytes instead of one, and all of the 55 bytes form the key, which gives a total of 2^{440} possible keys, and this number is so large that it would be practically impossible to try all of them.

3 Design

Our implementation included a separate encoder and decoder program for each generator, which makes 6 total programs, plus another program for the brute-force attack. To make them easier to use, there is a small menu program, `main.c`, which allows the user to choose which program to use, and it compiles the program and runs it.

All of the generator values are stored in the `uint8_t` type, which holds exactly one byte, so because of this, every result that goes above 255 wraps around automatically, so we did not have to implement the "wrap around" feature for the LCG and the SG methods.

The encoders all follow the same basic steps:

- First, the key is read, which will be the seed for the random generator, and for LCG, the seed together with a and c is used, whereas for the SG, the 55 initial bytes are used.
- Then, the message is read as a whole line with `fgets`, and the newline at the end is removed, and the length of the message is found with `strlen`.
- Using the length of the message, the code goes through every character of the message, and the generator produces a new byte, and the character is combined with it using the XOR operator.
- The length and the encoded bytes are printed as plain numbers between 0-255. We print the numbers instead of characters because not all encoded bytes are printable, and if there happened to be a byte with a zero value, the string would be cut off early.

And the decoders work in a similar way:

- First, the same key is read together with the length of the message and the encoded bytes.
- Then, the generator is restarted from the key, so it produces the exact same bytes as during encoding, and they are combined with the encoded message again using the XOR operation.
- And one extra byte is reserved for the special ending character, so it can be printed as a normal string.

For the SG, the last 55 bytes are kept in an array, where the first element is the oldest byte, and the element at index 31 is the $(n-24)$th byte. A new byte is calculated by subtracting the first element from the one at index 31, and then every element is moved one place to the left, so the new byte can be placed at the end.

The brute-force algorithm works in this way:

- First, it reads the type of generator, the encoded message, and a small part of the original message that the attacker is assumed to know.
- Then, it tries all the 256 possible seeds, decoding the whole message with each possible seed, checking if any part of the message matches the small part of the original message.
- Every seed that then matches is printed together with the decoded message, and the program also measures the time it took to complete the search.

For the LCG, we assume that a and c are already known, so the only part that needs to be guessed is the seed, whereas if a and c were unknown, we would have to repeat the same search for each possible value of them.

4 Program code

random_encode.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    int i;
    uint8_t seed, x; // generator's variables

    // reading the input
    printf("Input seed: ");
    scanf("%hhu", &seed);

    int n = 1000; // max lenght of the sequence
    char* message = calloc(n, sizeof(char));

    fgets(message, n, stdin);
    printf("Input message: ");
    message = fgets(message, n, stdin);
    message[strcspn(message, "\n")] = '\0';
    n = strlen(message);

    char* encMessage = calloc(n, sizeof(char));

    // message encoding
    srand(seed);
    for (int i = 0; i < n; i++) {
        // generating a new byte
        x = rand() % 256;
        encMessage[i] = (message[i] ^ x);
    }

    printf("Encoded message size: %d\n", n);
    printf("Encoded message bytes:\n");
    for (i = 0; i < n; i++) {
        printf("%hhu ", encMessage[i]);
    }

    return 0;
}
```

random_decode.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    int i;
    uint8_t seed, x; // generator's variables

    // reading the input
    printf("Input seed: ");
    scanf("%hhu", &seed);

    int n; // lenght of the sequence
```

```
14      printf("Input encoded size: ");
15      scanf("%d", &n);
16
17      char* encMessage = calloc(n, sizeof(char));
18      printf("Input encoded message bytes: ");
19      for (i = 0; i < n; i++) {
20          scanf("%hhu", &encMessage[i]);
21      }
22
23      char* decMessage = calloc(n + 1, sizeof(char)); // reserve an extra byte for the end
            of the string marker(\0)
24
25      // message decoding
26      srand(seed);
27      for (i = 0; i < n; i++) {
28          // generating a new byte
29          x = rand() % 256;
30          decMessage[i] = (encMessage[i] ^ x);
31      }
32
33      printf("Decoded message:\n%s\n", decMessage);
34
35      return 0;
36  }
```

lcg_encode.c

```
1   #include <stdio.h>
2   #include <stdlib.h>
3   #include <string.h>
4
5   int main(void) {
6       int i;
7       // generator's variables
8       uint8_t a;
9       uint8_t c;
10      uint8_t x;
11      uint8_t seed;
12
13      printf("Input seed: ");
14      scanf("%hhu", &seed);
15
16      printf("Input a: ");
17      scanf("%hhu", &a);
18
19      printf("Input c: ");
20      scanf("%hhu", &c);
21
22      int n = 1000; // max lenght of the sequence
23      char* message = calloc(n, sizeof(char));
24
25      fgets(message, n, stdin);
26      printf("Input message: ");
27      message = fgets(message, n, stdin);
28      message[strcspn(message, "\n")] = '\0';
29      n = strlen(message);
30
31      char* encMessage = calloc(n, sizeof(char));
```

```
32
33      // message encoding
34      x = seed;
35      for (int i = 0; i < n; i++) {
36          // generating a new byte
37          x = a * x + c;
38          encMessage[i] = (message[i] ^ x);
39      }
40
41      printf("Encoded message size: %d\n", n);
42      printf("Encoded message bytes:\n");
43      for (i = 0; i < n; i++) {
44          printf("%hhu ", encMessage[i]);
45      }
46
47      return 0;
48  }
```

lcg_decode.c

```
1   #include <stdio.h>
2   #include <stdlib.h>
3   #include <string.h>
4
5   int main(void) {
6       int i;
7       // generator's variables
8       uint8_t a;
9       uint8_t c;
10      uint8_t x;
11      uint8_t seed;
12
13      printf("Input seed: ");
14      scanf("%hhu", &seed);
15
16      printf("Input a: ");
17      scanf("%hhu", &a);
18
19      printf("Input c: ");
20      scanf("%hhu", &c);
21
22      int n; // lenght of the sequence
23      printf("Input encoded size: ");
24      scanf("%d", &n);
25
26      char* encMessage = calloc(n, sizeof(char));
27      printf("Input encoded message bytes: ");
28      for (i = 0; i < n; i++) {
29          scanf("%hhu", &encMessage[i]);
30      }
31
32      char* decMessage = calloc(n + 1, sizeof(char)); // reserve an extra byte for the end
            of the string marker(\0)
33
34      // message decoding
35      x = seed;
36      for (i = 0; i < n; i++) {
37          // generating a new byte
```

```
        x = a * x + c;
        decMessage[i] = (encMessage[i] ^ x);
    }

    printf("Decoded message:\n%s\n", decMessage);

    return 0;
}
```

sg_encode.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    int i;
    // generator's variables
    uint8_t* keys = calloc(55, sizeof(uint8_t));
    uint8_t* xs = calloc(55, sizeof(uint8_t));
    uint8_t x, r;

    printf("Input 55 keys: ");
    for (i = 0; i < 55; i++) {
        scanf("%hhu", &keys[i]);
        xs[i] = keys[i];
    }

    int n = 1000; // max lenght of the sequence
    char* message = calloc(n, sizeof(char));

    printf("Input message: ");
    fgets(message, n, stdin);
    message = fgets(message, n, stdin);
    message[strcspn(message, "\n")] = '\0';
    n = strlen(message);

    char* encMessage = calloc(n, sizeof(char));

    // message encoding
    for (i = 55; i < n + 55; i++) {
        // generating a new byte
        x = xs[31] - xs[0];
        for (int j = 0; j < 54; j++) {
            xs[j] = xs[j + 1];
        }
        xs[54] = x;
        encMessage[i - 55] = (message[i - 55] ^ x);
        // printf("%d ",x);
    }

    printf("Encoded message size: %d\n", n);
    printf("Encoded message bytes:\n");
    for (i = 0; i < n; i++) {
        printf("%hhu ", encMessage[i]);
    }
    return 0;
}
```

sg_decode.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    int i;
    // generator's variables
    uint8_t* keys = calloc(55, sizeof(uint8_t));
    uint8_t* xs = calloc(55, sizeof(uint8_t));
    uint8_t x, r;

    printf("Input 55 keys: ");
    for (i = 0; i < 55; i++) {
        scanf("%hhu", &keys[i]);
        xs[i] = keys[i];
    }

    int n; // lenght of the sequence
    printf("Input encoded size: ");
    scanf("%d", &n);

    char* encMessage = calloc(n, sizeof(char));
    printf("Input encoded message bytes: ");
    for (i = 0; i < n; i++) {
        scanf("%hhu", &encMessage[i]);
    }

    char* decMessage = calloc(n + 1, sizeof(char));

    // message decoding
    for (i = 0; i < 55; i++) {
        xs[i] = keys[i];
    }
    for (i = 55; i < n + 55; i++) {
        // generating a new byte
        x = xs[31] - xs[0];
        for (int j = 0; j < 54; j++) {
            xs[j] = xs[j + 1];
        }
        xs[54] = x;
        decMessage[i - 55] = (encMessage[i - 55] ^ x);
    }

    printf("Decoded message: %s\n", decMessage);

    return 0;
}
```

brute_force.c

```c
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

int main(void) {
```

```
    int method, a, c, n, size, value;
    unsigned char encrypted[1000];
    unsigned char decoded[1000];
    unsigned char known[1000];

    printf("Input method (1 = rand, 2 = lcg), a and c (use 0 0 for rand): ");
    if (scanf("%d %d %d", &method, &a, &c) != 3) return 1;
    if (method < 1 || method > 2 || a < 0 || a > 255 || c < 0 || c > 255) return 1;
    printf("Input encoded size: ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 999) return 1;

    // encoded message input
    printf("Input encoded message numerical: ");
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &value) != 1 || value < 0 || value > 255) return 1;
        encrypted[i] = value;
    }

    // known part of the message
    printf("Input known message size: ");
    if (scanf("%d", &size) != 1 || size < 1 || size > n) return 1;
    printf("Input known message numerical: ");
    for (int i = 0; i < size; i++) {
        if (scanf("%d", &value) != 1 || value < 0 || value > 255) return 1;
        known[i] = value;
    }

    // try every seed
    printf("\nMatching seeds and decoded message bytes:\n");
    clock_t start = clock();
    for (int seed = 0; seed <= 255; seed++) {
        uint8_t x = seed;
        srand(seed);

        // message decoding
        for (int i = 0; i < n; i++) {
            if (method == 1) {
                x = rand() % 256;
            } else {
                x = a * x + c;
            }
            decoded[i] = encrypted[i] ^ x;
        }

        // check for a match
        for (int i = 0; i <= n - size; i++) {
            if (memcmp(decoded + i, known, size) == 0) {
                printf("%d", seed);
                for (int j = 0; j < n; j++) {
                    printf(" %u", (unsigned int)decoded[j]);
                }
                printf("\n");
                break;
            }
        }
    }

    double seconds = (double)(clock() - start) / CLOCKS_PER_SEC;
```

```
    printf("CPU time for 256 seeds: %.6f seconds\n", seconds);

    return 0;
}
```

main.c

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    const char* programs[] = {
        "random_encode", "random_decode",
        "lcg_encode", "lcg_decode",
        "sg_encode", "sg_decode",
        "brute_force"
    };
    int choice;
    char input[100], extra;
    char command[512];

    setvbuf(stdin, NULL, _IONBF, 0);

    printf("ENIGMA++\n");
    printf("1. random encode\n2. random decode\n");
    printf("3. lcg encode\n4. lcg decode\n");
    printf("5. sg encode\n6. sg decode\n");
    printf("7. brute force\n");
    printf("Choose an option: ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL) return 1;
    if (sscanf(input, "%d %c", &choice, &extra) != 1 || choice < 1 || choice > 7) {
        printf("Invalid option.\n");
        return 1;
    }

    const char* name = programs[choice - 1];
    snprintf(command, sizeof(command),
        "mkdir -p build && cc -std=c11 -Wall -Wextra -pedantic c/%s.c -o build/%s && ./
            build/%s",
        name, name, name);

    if (system(command) != 0) {
        printf("Could not compile or run the program. Run this menu from the project
            folder.\n");
        return 1;
    }

    return 0;
}
```

5 Test results

Random encryption/decryption

Encryption:

- Input: (with additional prints for the user)

```
Input seed: 42
Input message: Hello! This is an unencrypted message.
```

Output:

```
Encoded message size: 38
Encoded message bytes:
46 234 213 155 5 248 81 156 114 160 115 39 115 102 64 221 140 234 117 164 162 136 25
     139 234 56 229 82 5 99 45 113 168 44 83 208 157 111
```

Decryption:

- Input: (with additional prints for the user)

```
Input seed: 42
Input encoded size: 38
Input encoded message bytes: 46 234 213 155 5 248 81 156 114 160 115 39 115 102 64
    221 140 234 117 164 162 136 25 139 234 56 229 82 5 99 45 113 168 44 83 208 157
    111
```

Output:

```
Decoded message:
Hello! This is an unencrypted message.
```

LCG encryption/decryption

Encryption:

- Input: (with additional prints for the user)

```
Input seed: 42
Input a: 5
Input c: 10
Input message: Hello! This is an unencrypted message.
```

Output:

```
Encoded message size: 38
Encoded message bytes:
148 51 212 206 91 47 112 206 100 47 27 50 13 141 32 107 82 22 109 236 241 128 211 8
    21 86 188 151 160 254 13 143 239 101 25 5 145 224
```

Decryption:

- Input: (with additional prints for the user)

```
Input seed: 42
Input a: 5
Input c: 10
Input encoded size: 38
Input encoded message bytes: 148 51 212 206 91 47 112 206 100 47 27 50 13 141 32 107
     82 22 109 236 241 128 211 8 21 86 188 151 160 254 13 143 239 101 25 5 145 224
```

Output:

```
Decoded message:
Hello! This is an unencrypted message.
```

SG encryption/decryption

Encryption:

- Input: (with additional prints for the user)

```
Input 55 keys: 102 143 185 247 106 217 113 200 26 201 0 7 26 21 96 188 226 202 0 202
     199 230 122 249 147 72 145 55 97 67 64 20 219 95 50 183 248 65 231 161 160 9
    104 239 253 237 18 115 95 196 122 62 177 59 87
Input message: Hello! This is an unencrypted message.
```

Output:

```
Encoded message size: 38
Encoded message bytes:
230 41 202 87 34 62 240 75 239 190 122 65 188 155 173 55 255 181 177 222 18 165 162
    44 98 116 97 97 136 252 253 110 223 11 182 205 184 137
```

Decryption:

- Input: (with additional prints for the user)

```
Input 55 keys: 102 143 185 247 106 217 113 200 26 201 0 7 26 21 96 188 226 202 0 202
     199 230 122 249 147 72 145 55 97 67 64 20 219 95 50 183 248 65 231 161 160 9
    104 239 253 237 18 115 95 196 122 62 177 59 87
Input encoded size: 38
Input encoded message bytes: 230 41 202 87 34 62 240 75 239 190 122 65 188 155 173
    55 255 181 177 222 18 165 162 44 98 116 97 97 136 252 253 110 223 11 182 205 184
     137
```

Output:

```
Decoded message:
Hello! This is an unencrypted message.
```

Brute-force attack on the random generator

We assume that the attacker knows that the message contains the word Hello (72 101 108 108 111).

- Input: (with additional prints for the user)

```
Input method (1 = rand, 2 = lcg), a and c (use 0 0 for rand): 1 0 0
Input encoded size: 38
Input encoded message numerical: 46 234 213 155 5 248 81 156 114 160 115 39 115 102
    64 221 140 234 117 164 162 136 25 139 234 56 229 82 5 99 45 113 168 44 83 208
    157 111
Input known message size: 5
Input known message numerical: 72 101 108 108 111
```

Output:

```
Matching seeds and decoded message bytes:
42 72 101 108 108 111 33 32 84 104 105 115 32 105 115 32 97 110 32 117 110 101 110
    99 114 121 112 116 101 100 32 109 101 115 115 97 103 101 46
CPU time for 256 seeds: 0.000189 seconds
```

Brute-force attack on the LCG

- Input: (with additional prints for the user)

```
Input method (1 = rand, 2 = lcg), a and c (use 0 0 for rand): 2 5 10
Input encoded size: 38
Input encoded message numerical: 148 51 212 206 91 47 112 206 100 47 27 50 13 141 32
     107 82 22 109 236 241 128 211 8 21 86 188 151 160 254 13 143 239 101 25 5 145
    224
Input known message size: 5
Input known message numerical: 72 101 108 108 111
```

Output:

```
Matching seeds and decoded message bytes:
42 72 101 108 108 111 33 32 84 104 105 115 32 105 115 32 97 110 32 117 110 101 110
    99 114 121 112 116 101 100 32 109 101 115 115 97 103 101 46
CPU time for 256 seeds: 0.000112 seconds
```

In both cases, the only matching seed is 42, and the decoded bytes are the original message.

6 Evaluation

We have designed the programs to be user-friendly through interactions in the console. This way, we were able to easily test the encryption and decryption algorithms for each generator. By selecting the `uint8_t` datatype for storing the bytes, the programs became faster thanks to the automatic modulus operation executed by the overflow.

Considering the study on the number of brute-force attacks to break the code (2^8 tries for the Random Generator, 2^{24} tries for Linear Congruential Generator and 2^{440} tries for the Subtractive Generator), we can conclude that the first two can be very quickly brute-forced, whereas the third one is impossible.

This is also confirmed by our tests, where the brute-force program found the correct seed and the whole message in less than a millisecond for both the random generator and the LCG. If a and c of the LCG are also unknown, the search then becomes 2^{16} times longer, but that would still take only around 10 seconds.

Trying all 256 seeds took 0.0002 seconds, which is around 0.0000007 seconds per key. The subtractive generator has 2^{440} possible keys, which is 2^{432} times more than the first two, so trying all of the combinations would take around $2 \cdot 10^{126}$ seconds, which is about $7 \cdot 10^{118}$ years.